

(Project Title)
(Team Name and Number)
(Team Members)

[Project Documentation]

Version: (n)

Date: (mm/dd/yyyy)

Table of Contents

Chapter 1 Introduction	4
1.1 Purpose	4
1.2 Scope	4
1.3 Definitions, Acronyms, and Abbreviations.	4
1.4 References	4
1.5 Overview	5
1.6 Product Perspective	5
1.6.1 System Interfaces	5
1.6.2 Interfaces	6
1.6.3 Hardware Interfaces	6
1.6.4 Software Interfaces	6
1.6.5 Communications Interfaces	7
1.6.6 Memory Constraints	7
1.6.7 Operations	8
1.6.8 Site Adaptation Requirements	8
1.7 Product Functions	8
1.8 User Characteristics	9
1.9 Constraints	9
1.10 Assumptions and Dependencies	10
1.11 Apportioning of Requirements	10
Chapter 2 Software Requirements Specification	11
2.1 Specific Requirements	11
2.2 External Interfaces	12
2.3 Functions	13
2.4 Performance Requirements	13
2.5 Logical Database Requirements	14
2.6 Design Constraints	14
2.6.1 Standards Compliance	14
2.7 Software System Attributes	15
2.7.1 Reliability	15
2.7.2 Availability	15
2.7.3 Security	16
2.7.4 Maintainability	16
2.7.5 Portability	16
2.8 Organizing the Specific Requirements	17
2.8.1 System Mode	18
2.8.2 User Class	18
2.8.3 Objects	18
2.8.4 Feature	18
2.8.5 Stimulus	18
2.8.6 Response	18

2.8.7 Functional Hierarchy	18
2.9 Additional Comments	19
Chapter 3 System Design and Architecture	20
3.1 Architectural Design	20
3.2 Decomposition Description	20
3.3 Design Rationale	21
3.4 Data Description	21
3.5 Data Dictionary	21
3.6 COMPONENT DESIGN	21
Chapter 4 Implementation and Testing	22
4.1 Code Modules	22
4.2 Database Connectivity	22
4.3 Overview of User Interface	22
4.4 Screen Images	22
4.5 Screen Objects and Actions	22
4.6 Test Cases	23

Chapter 1

Introduction

The following subsections of the Software Requirements Specifications (SRS) document should provide an overview of the entire SRS. The thing to keep in mind as you write this document is that you are telling what the system must do – so that designers can ultimately build it. Do not use this document for design!!!

1.1 Purpose

This Software Requirements Specification (SRS) and Software Design Description (SDD) document specifies the complete software requirements and design for the Campus Navigation System Web Project developed for The Islamia University of Bahawalpur (IUB). This document ensures that the development team and stakeholders have a clear, unambiguous understanding of the system's functionality, design, and constraints.

The purpose of this document is to serve as a comprehensive guide for implementation and testing. It bridges the gap between the high-level project vision and the detailed technical work required to build the system. The intended audience includes the project supervisor, development team, testers, and stakeholders.

1.2 Scope

(1) The software product is named "Campus Navigation System" – a web-based application for The Islamia University of Bahawalpur.

(2) The system will enable users to:

1. View a detailed interactive map of the university campus
2. Search for specific locations including buildings, departments, and Points of Interest (POIs)
3. Calculate and display the shortest walking route between any two selected points
4. View distance in meters and estimated walking time in minutes

What the system will NOT do in its initial version:

1. Indoor navigation within buildings is excluded

2. Real-time traffic or congestion updates are not included
3. Native mobile application is not included
4. Integration with university student information systems is not included

(3) The application replaces manual direction-finding methods (paper maps, asking for directions), improving accessibility for students, faculty, and visitors.

(4) This document is consistent with the approved project proposal submitted to the Department of Computer Science, IUB.

1.3 Definitions, Acronyms, and Abbreviations.

1. POI - Point of Interest (building, department, cafeteria, facility)
2. SRS - Software Requirements Specification
3. SDD - Software Design Description
4. PostGIS - Spatial database extension for PostgreSQL
5. API - Application Programming Interface
6. ORM - Object-Relational Mapper
7. OOP - Object-Oriented Programming
8. IUB - The Islamia University of Bahawalpur
9. JSON - JavaScript Object Notation
10. REST - Representational State Transfer
11. FR - Functional Requirement
12. NFR - Non-Functional Requirement.

1.4 References

1. IUB Campus Master Plan – Official map and spatial data from IUB Estate Office
2. OpenStreetMap Documentation – <https://wiki.openstreetmap.org>
3. Django Documentation – <https://docs.djangoproject.com>
4. PostgreSQL/PostGIS Documentation – <https://postgis.net>
5. Software Engineering - A Practitioner's Approach – Roger S. Pressman
6. Software Engineering 9th Edition – Ian Sommerville
7. Agile Modeling – <http://agilemodeling.com/artifacts/>

1.5 Overview

Chapter 2 presents the Software Requirements Specification including specific requirements, external interfaces, functions, performance requirements, and software system attributes. Customers should focus on this chapter.

Chapter 3 presents System Design and Architecture including architectural design, decomposition, data design, and component design with pseudo code. Developers should focus on this chapter.

Chapter 4 presents Implementation and Testing including code modules, database connectivity, user interface, screen images, and test cases.

1.6 Product Perspective

The Campus Navigation System is an entirely new, independent product. It does not replace any existing software system because no such navigation system currently exists at IUB. Students, faculty, and visitors currently rely on paper maps or asking for directions.

The system operates within the larger context of university IT infrastructure but does not depend on any existing software systems. It is a standalone web application that can be deployed on university servers or cloud hosting.

1.6.1 System Interfaces

The system has no external system interfaces in its initial version. It operates independently and does not require integration with Student Information Systems, Learning Management Systems, or other university enterprise systems.

1.6.2 Interfaces

1. The system provides a graphical user interface (GUI) accessible through standard web browsers. Key interface elements include:
2. Interactive Map – Users can zoom, pan, and click on locations
3. Search Bar – Text input with auto-complete functionality
4. "Get Directions" Button – Triggers route calculation
5. Route Information Panel – Displays distance and estimated time
6. Secure Admin Panel – Separate login page for administrators
7. Admin Forms – For adding, editing, and deleting POIs and paths

1.6.3 Hardware Interfaces

The system has no direct hardware interface requirements. It is a purely software-based web application running on standard web servers and accessed through standard user devices.

1.6.4 Software Interfaces

The system interfaces with three main software components:

PostgreSQL with PostGIS Database – Stores all spatial and non-spatial data including map geometry, POIs, paths, and admin user information. Uses spatial functions like ST_Distance, ST_Length, and ST_AsGeoJSON.

Django Web Framework with ORM – Provides backend application logic, request handling, and database abstraction. Manages models, views, and URL routing.

REST API (Django REST Framework) – Enables frontend-backend communication through RESTful endpoints returning JSON data. Example: GET /api/route?start=A&end=B returns the shortest path as GeoJSON.

1.6.5 Communications Interfaces

The system uses standard HTTP/HTTPS protocols over TCP/IP networks. All data exchanged between client and server uses JSON format for structured data and GeoJSON for spatial data. HTTPS is used for all communications in production.

1.6.6 Memory Constraints

The system has no specific memory constraints beyond standard web hosting requirements. Server-side recommendations include 4 GB RAM minimum for PostgreSQL/PostGIS and 2 GB RAM for Django. Client devices require only standard web browser memory allocation.

1.6.7 Operations

The system operates in two modes: Public User Mode (map viewing, search, route calculation) available 24/7 without authentication, and Administrative Mode (data management) requiring secure login. Database backups are scheduled weekly by the system administrator.

1.6.8 Site Adaptation Requirements

1. PostgreSQL with PostGIS must be installed before system installation
2. SSD-backed volume required for database to meet performance targets
3. Linux-based OS (Ubuntu 20.04+) recommended for production server
4. IUB Campus Master Plan data must be imported into the database

1.7 Product Functions

The major functions of the software are:

F-01: Interactive Map Display – The system displays a navigable campus map allowing users to zoom in, zoom out, and pan across different areas of the university.

F-02: Shortest Route Computation – The system calculates the shortest pedestrian route between any two user-selected points using Dijkstra's algorithm implemented in PostGIS.

F-03: Route Statistics Display – The system displays total distance in meters and estimated walking time in minutes (assuming average walking speed of 1.4 m/s or 5 km/h).

F-04: Location Search – The system provides text-based search with auto-complete to help users quickly find specific buildings, departments, or Points of Interest.

F-05: POI Selection – Users can click directly on map markers to select them as start or destination points.

F-06: Admin Authentication – The system provides secure login for administrators to access data management functions.

F-07: POI Data Management – Authorized administrators can Add, Edit, and Delete Points of Interest including name, type, and geographic coordinates.

F-08: Path Data Management – Authorized administrators can Add, Edit, and Delete walking paths between nodes on campus.

F-09: Route Visualization – The calculated route is visually highlighted on the interactive map with a distinctive blue line.

1.8 User Characteristics

The intended users have been analyzed to understand their characteristics:

Students (age 18-25) – Digital natives, comfortable with web and mobile apps, expect fast responses. Design implication: Modern UI, mobile-responsive design, minimal instructions.

Faculty (age 30-60) – Varying technical expertise; some may be first-time users. Design implication: Simple, intuitive interface with clear labels and help text.

Visitors (all ages) – No prior experience with the system; unfamiliar with campus layout. Design implication: Very simple interface with prominent search and directions and clear visual cues.

Administrators (age 25-45) – Responsible for data accuracy. Design implication: Efficient data entry forms with confirmation dialogs and action logging.

The interface must be intuitive enough for a first-time visitor to use without help. Text must be readable (minimum 12px font), buttons clearly labeled with common terms, and error messages helpful and instructional.

1.9 Constraints

1. The following constraints limit developer options during design and implementation:
2. Regulatory Policies – The system must comply with university data privacy policies
3. Audit Functions – All Admin actions (add, update, delete) must be logged with timestamp and admin ID
4. Control Functions – Only authorized Administrators can modify campus map data
5. Language Requirements – All backend code in Python 3.9+ adhering to PEP 8 style guide
6. Database Constraint – Must use PostgreSQL with PostGIS extension
7. Data Source Constraint – Must rely solely on IUB Campus Master Plan data; no commercial map APIs for routing
8. Licensing Constraint – All third-party libraries must use open-source licenses (MIT, GPL, or BSD)
9. Availability Constraint – System must be accessible 24/7.

1.10 Assumptions and Dependencies

1. The university will provide access to IUB Campus Master Plan spatial data
2. PostgreSQL database server with PostGIS extension will be available
3. Users will have modern web browsers and active internet connections
4. The system relies solely on geometric data from IUB Campus Master Plan

5. The hosting server will have at least 99% uptime
6. Administrators will keep map data up-to-date and accurate

1.11 Apportioning of Requirements

1. The following requirements are for future versions (NOT in Version 1.0):
2. Indoor Navigation – Version 2.0
3. Native Mobile Application – Version 2.0
4. Real-time Path Closure Updates – Version 2.0
5. Turn-by-Turn Voice Navigation – Version 2.0
6. Integration with University Event Calendar – Version 2.0
7. Accessibility Routes (wheelchair friendly) – Version 2.0
8. Multiple Language Support (Urdu, etc.) – Version 2.0

Chapter 2

Software Requirements Specification

2.1 Specific Requirements

The following specific requirements are stated with complete, unambiguous, verifiable characteristics. Each requirement is uniquely identifiable using the FR prefix.

FR-1: Interactive Map Display (Priority: High)

1. The system shall display a navigable map of IUB campus as the main interface
2. The system shall allow users to zoom in and out using buttons or mouse wheel
3. The system shall allow users to pan across the map by clicking and dragging
4. The system shall display the map within 5 seconds of page load

FR-2: Shortest Route Computation (Priority: High)

1. The system shall allow users to select start and destination points by clicking on the map
2. The system shall calculate the shortest walking route between selected points
3. The system shall use Dijkstra's shortest path algorithm implemented in PostGIS
4. The system shall complete 95% of route calculation requests within 3 seconds

FR-3: Route Statistics (Priority: High)

1. The system shall display total distance of the calculated route in meters
2. The system shall calculate estimated walking time based on average walking speed (1.4 m/s)
3. The system shall display estimated walking time in minutes

FR-4: Location Search (Priority: Medium)

1. The system shall provide a text input field for searching locations
2. The system shall provide auto-complete suggestions as the user types within 500 milliseconds
3. When a search result is selected, the system shall highlight the POI on the map

FR-5: Data Management - Admin (Priority: High)

1. The system shall provide a secure login interface for administrators
2. The system shall allow Admin to add, edit, and delete Points of Interest
3. The system shall allow Admin to add, edit, and delete walking paths
4. The system shall confirm all operations before saving and provide feedback

FR-6: Route Visualization (Priority: Medium)

1. The system shall display the calculated route as a colored line on the map
2. The system shall display start and end points with distinct markers

2.2 External Interfaces

User Interface Inputs:

The system accepts the following inputs from users:

1. Search Keyword – Text entered by user (1-100 characters, alphanumeric) to find POIs
2. Start Point Selection – Mouse click on map selecting valid coordinates
3. Destination Selection – Mouse click on map selecting valid coordinates
4. Username/Password – Admin login credentials (3-50 characters)
5. POI Data – Name, type, latitude, longitude entered by admin

User Interface Outputs:

The system produces the following outputs:

1. Interactive Map – Campus map visualization displayed within 5 seconds
2. Route Line – Visual path between points accurate within 2-3 meters, displayed within 3 seconds
3. Distance Value – Calculated route distance in meters (accuracy $\pm 2\%$)
4. Time Value – Estimated walking time in minutes (accuracy ± 1 minute)
5. Search Results – List of matching POIs in dropdown (100% accuracy)
6. Error Messages – Alerts for invalid inputs or unavailable routes
7. Confirmation Messages – Success feedback for admin actions

2.3 Functions

The system shall perform the following functions:

F-01: Map Loading Function – Fetch map tiles from tile server, render base map layer, overlay POI markers, initialize zoom and pan controls.

F-02: Input Validation Functions – Validate start and end points exist in database, validate points are connected (route exists), validate admin credentials, validate POI data before saving.

F-03: Route Calculation Sequence:

1. User selects start point → System displays start marker
2. User selects destination → System displays destination marker
3. User clicks "Get Directions" → System validates both points
4. If validation fails → Error message displayed
5. If validation passes → Backend fetches path graph from PostGIS
6. Backend applies Dijkstra's algorithm → Calculates distance and time
7. Backend returns route geometry as GeoJSON
8. Frontend displays route on map and statistics in info panel

F-04: Error Handling Responses:

1. Please select valid points on the map" – for invalid start/end
2. No walking route found between these points" – for disconnected points
3. No matching locations found" – for search with no results
4. Invalid username or password" – for failed admin login
5. System temporarily unavailable" – for database connection failure

2.4 Performance Requirements

Static Numerical Requirements:

1. The system shall support unlimited simultaneous users (web-based)
2. The system shall handle 25 concurrent route calculation requests per second

Dynamic Numerical Requirements:

1. 95% of route calculation requests shall be processed within 3 seconds
2. The interactive map shall fully load within 5 seconds
3. Auto-complete search suggestions shall appear within 500 milliseconds
4. The system shall sustain 25 concurrent route calculation requests per second

Capacity Requirements:

1. Handle peak demand during student orientation week (500+ simultaneous users)
2. Store at least 500 POIs without performance degradation
3. Store at least 1000 path edges without performance degradation

2.5 Logical Database Requirements

Types of Information Used:

1. POI (Points of Interest) – Building names, types, coordinates (~500 bytes per POI)
2. Path Edges – Connections between nodes with geometry (~200 bytes per edge)
3. Nodes – Intersection points (~100 bytes per node)
4. Admin Users – Username and password hash (~200 bytes per admin)

Frequency of Use:

1. Path graph: Very high (per route request)
2. POI data: High (per search query)
3. Map geometry: High (per map load)
4. Admin user data: Low (per login attempt)

Integrity Constraints:

1. Every Path edge must connect to two valid existing Nodes
2. Every Path distance must be positive (> 0)
3. Every POI must have non-empty name and valid coordinates
4. No duplicate POI names allowed
5. Admin username must be unique

Data Retention Requirements:

1. POI data: Permanent (until deleted by admin)
2. Path data: Permanent (until deleted by admin)
3. Admin accounts: Permanent (until deleted)
4. Route results: Temporary (not stored)
5. Audit logs: Minimum 30 days

2.6 Design Constraints

The following design constraints are imposed on the development team:

1. Backend language – Python 3.9+ with PEP 8 compliance
2. Web framework – Django 3.2+
3. Database – PostgreSQL with PostGIS extension
4. Mapping library – Open-source (Leaflet/OpenLayers) – no commercial licenses
5. API style – RESTful with JSON
6. No commercial map APIs for routing – customer requirement
7. Admin authentication required for all data modification

2.6.1 Standards Compliance

Report Format Standards: All API responses must be in JSON format following RESTful conventions. Geographic data must be returned as GeoJSON where applicable.

Data Naming Standards: Database tables and columns use snake_case. API endpoints use lowercase with underscores or hyphens.

Audit Tracing Standards: All Admin actions logged with timestamp, admin username, action type, affected data, and before/after values for UPDATE actions.

Security Standards: Passwords hashed using bcrypt or PBKDF2. All communications use HTTPS in production. Admin sessions expire after 30 minutes of inactivity.

2.7 Software System Attributes

There are a number of attributes of software that can serve as requirements. It is important that required attributes are specified so that their achievement can be objectively verified. The following items provide a partial list of examples. These are also known as non-functional requirements or quality attributes.

These are characteristics the system must possess, but that pervade (or cross-cut) the design. These requirements have to be testable just like the functional requirements.

2.7.1 Reliability

The system shall handle invalid inputs gracefully without crashing. It shall continue to function for read operations even if database write operations are slow. System uptime target is 99.5% (excluding planned maintenance).

2.7.2 Availability

The system must be available 24 hours per day, 7 days per week. Maximum planned downtime is 2 hours per week for maintenance during off-peak hours (2 AM - 4 AM). Maximum unplanned downtime is 4 hours per month. The system shall allow users to restart after failure with no data loss.

2.7.3 Security

Only authorized Administrators can change or delete campus map data. Authentication is required for all admin endpoints. Passwords are stored using cryptographic hashing (bcrypt). No public access to admin functions. Admin sessions use secure, HTTP-only cookies. All admin actions are logged for audit purposes.

2.7.4 Maintainability

Code follows Object-Oriented Design principles with modular structure and clear separation of concerns (MVC pattern). Comments are present for all non-obvious functions and logic. All third-party libraries use open-source licenses. Database schema and API endpoints are documented.

2.7.5 Portability

The system runs on any device supporting modern web browsing (Windows, macOS, Linux, iOS, Android). It operates correctly with Google Chrome, Mozilla Firefox, Microsoft Edge, and Apple Safari (latest two versions). The interface is responsive on screen sizes from 320px (mobile) to 1920px (desktop). No browser plugins or extensions are required.

Quality Characteristics Priority (High to Low):

1. Correctness – H
2. Efficiency – H
3. Usability – H
4. Security – H
5. Availability – H
6. Maintainability – M
7. Reliability – M
8. Portability – M
9. Testability – M

- 10. Flexibility – L
- 11. Interoperability – L
- 12. Reusability – L

2.8 Organizing the Specific Requirements

For anything but trivial systems, the detailed requirements tend to be extensive. For this reason, careful consideration has been given to organizing these requirements in a manner optimal for understanding. The Campus Navigation System requirements are organized using multiple complementary approaches as described in the following subsections.

2.8.1 System Mode

The system behaves differently depending on the mode of operation:

Public User Mode (Patron):

- 1. Display interactive campus map
- 2. Search for POIs and view results
- 3. Select start and destination points
- 4. Request and view routes
- 5. No authentication required

Administrative Mode (Admin):

- 1. Secure login required
- 2. Add, edit, and delete POIs
- 3. Add, edit, and delete paths
- 4. All actions logged for audit

2.8.2 User Class

Different functions are provided to different classes of users:

Patron Class (Students, Faculty, Visitors):

- 1. Read-only access to map and POI data
- 2. Functions: View map, search locations, calculate routes
- 3. Authentication: Not required

Admin Class (System Administrators):

- 1. Read and write access to all data

2. Functions: All patron functions plus POI and path management
3. Authentication: Required (username + password)

2.8.3 Objects

Real-world entities modeled within the system:

Entity	Attributes (Data)	Functions (Methods/Behavior)
Location	id, name, type, coordinates	searchLocation(), displayOnMap()
Route	id, start, end, distance, time	computeShortestRoute(), displayRoute()
User	user_id, session_id	searchLocation(), viewMap(), requestRoute()
Admin	admin_id, username, password_hash	login(), managePOI(), managePath()
Path	id, start_node, end_node, distance, geometry	calculateDistance(), getGeometry()

2.8.4 Feature

Externally desired services described as stimulus-response sequences:

Feature 1: Find Location

1. User types in search bar → System displays auto-complete suggestions
2. User selects suggestion → System pans to location and displays marker

Feature 2: Get Directions

1. User selects start point → System displays start marker
2. User selects destination → System displays destination marker
3. User clicks "Get Directions" → System displays route with distance and time

Feature 3: Manage Map Data (Admin)

1. Admin navigates to admin panel → System displays login form
2. Admin enters valid credentials → System displays data management interface
3. Admin adds/edits/deletes POI → System confirms and updates data

2.8.5 Stimulus

Inputs that trigger system behavior:

Stimulus	Source	Response
Load URL	Browser	Display map
Click zoom button	Mouse	Zoom in/out
Type in search bar	Keyboard	Show suggestions
Click on map	Mouse	Place marker
Click "Get Directions"	Mouse	Calculate route
Submit admin login	Keyboard/Mouse	Grant/deny access

Add/Edit/Delete POI

Admin mouse

Save or delete data

2.8.6 Response

Response (Output)	Destination (Target)	Purpose
Display interactive map	User screen	Base navigation interface provide karna.
Show auto-complete dropdown	User screen	Search results ko asaan banana.
Display marker at clicked location	User screen	Map par specific point highlight karna.
Show blue route line with distance and time	User screen	Path calculation ka visual result dena.
Display login form or dashboard	Admin screen	Management tools tak access dena.

Show success or error message	Admin screen	Actions (Save/Delete) ka feedback dena.
Redirect to login on session expiry	Admin screen	Security aur session management maintain karna.

2.8.7 Functional Hierarchy

The overall functionality is organized into a hierarchy of four levels:

Level 1: User Interface Layer

1. Render Map, Handle User Input, Display Search Results, Render Route, Display Admin Forms

Level 2: Application Logic Layer

1. Process Search Query, Validate Route Request, Compute Shortest Path (Dijkstra), Authenticate Admin, Process CRUD Operations

Level 3: Data Access Layer

1. Query POI Table, Query Path Graph, Insert/Update/Delete Data, Log Admin Actions

Level 4: Database Layer (PostGIS)

1. ST_Distance, ST_Length, ST_AsGeoJSON, ST_MakePoint

2.9 Additional Comments

A hybrid organizational approach combining system mode, user class, and feature-based organization is used for this SRS. This best suits the Campus Navigation System which has distinct public and administrative functions.

Recommended Notations:

1. State Charts – For modeling route calculation process (Idle → Start Selected → Destination Selected → Route Displayed)
2. Data Flow Diagrams – For showing data movement between user, application, and database
3. Class Diagrams – For showing object relationships (User, Admin, Location, Route, Path

Traceability:

All functional requirements (FR-1 to FR-5) are traceable to use cases, system modes, user classes, objects, features, stimuli, responses, and functions in the hierarchy.

Chapter 3

System Design and Architecture

3.1 Architectural Design

The Campus Navigation System follows a layered client-server architecture with three main layers. This architecture provides clear separation of concerns, allows independent development of frontend and backend, and supports scalability.

Layer 1: Presentation Layer (Frontend)

The presentation layer runs in the user's web browser and handles all user interactions and visual display.

Components:

1. Interactive Map Component – Renders campus map using Leaflet.js, handles zoom/pan, displays markers and routes
2. Search Interface Component – Accepts user text input, displays auto-complete suggestions, triggers POI search
3. Route Display Component – Renders route lines, displays distance and time statistics
4. Admin Panel Interface – Login form, POI management forms, path management forms

Technologies: HTML5, CSS3, JavaScript (ES6+), Leaflet.js, Fetch API

Layer 2: Application Layer (Backend)

The application layer runs on the web server and handles all business logic and request processing.

Components:

1. Route Calculation Service – Implements Dijkstra's algorithm, processes path graph, returns shortest path
2. Search Service – Handles POI search queries, returns matching locations
3. Authentication Service – Validates admin credentials, manages sessions
4. Admin Data Management Service – Processes add/edit/delete requests for POIs and paths
5. API Gateway – Exposes REST endpoints, validates requests, routes to appropriate services

Technologies: Python 3.9+, Django 3.2+, Django REST Framework

Layer 3: Data Layer (Database)

The data layer stores all persistent data for the system.

Components:

1. Spatial Data Store – PostGIS geometry types for paths, nodes, and POI locations
2. POI Data Store – Points of Interest information (name, type, coordinates)
3. Path Network Store – Edges, nodes, connections for the walking graph
4. Admin User Store – Admin credentials and session data

Technologies: PostgreSQL 13+, PostGIS 3.0+

Architecture Flow Diagram

text

[Web Browser] ↔ [Django Server] ↔ [PostgreSQL/PostGIS]



User Interface	Business Logic	Spatial Data
Map Display	Route Calculation	Path Graph
Search Form	Search Processing	POI Data
Admin Panel	Admin Functions	User Data

3.2 Decomposition Description

Aapke system ki complete decomposition aur process flow ko neeche tables mein organize kiya gaya hai:

Subsystem 1: Frontend Module (Client-Side)

Component	Responsibilities
Map Manager (map.js)	Map initialize karna, tile layers add karna, aur zoom/pan handle karna.
Marker Manager (markers.js)	POI markers ko add/remove karna aur clicks handle karna.
Route Renderer (route.js)	Map par route lines render karna aur purani routes clear karna.
Search Handler (search.js)	Input capture karna, suggestions fetch karna aur results dikhana.
Admin UI (admin.js)	Login handle karna, forms submit karna aur AJAX requests bhejna.
UI Controller (app.js)	Main coordinator jo events aur components ko manage karta hai.

Subsystem 2: Backend Services Module (Server-Side)

Component	Responsibilities
URL Router (urls.py)	URL patterns ko sahi view functions se map karna.
Route View (views_route.py)	Route requests handle karna aur route service ko call karna.
Search View (views_search.py)	Search requests handle karna aur POI table query karna.
Auth View (views_auth.py)	Login/Logout aur session management handle karna.
Admin View (views_admin.py)	POIs aur paths ke liye CRUD (Create, Read, Update, Delete) operations.
Route Algorithm	Dijkstra's algorithm ka use karke shortest path calculate karna.

Serializers (serializers.py)	Django models aur JSON data ke beech conversion karna.
-------------------------------------	--

Subsystem 3: Database Module (Data Storage)

Table	Description	Key Columns
auth_admin	Admin accounts ki details	id, username, password_hash
map_poi	Points of Interest ka data	id, name, type, latitude, longitude
map_node	Path network ke nodes	id, latitude, longitude
map_path	Walking paths ki details	id, start_node_id, end_node_id, distance, geometry
audit_log	Admin actions ka record	id, admin_id, action, timestamp

System Process Flow (Step-by-Step)

1. **Access:** User system access karta hai aur Frontend Map display karta hai.
2. **Input:** User search bar use karta hai ya map par points select karta hai.
3. **Validation:** Frontend input ko validate karke Backend ko request bhejta hai.
4. **Backend Processing:** Backend graph data fetch karta hai aur **Dijkstra Algorithm** run karta hai.
5. **Output:** Calculated route (distance aur time ke saath) map par display ho jati hai.

3.3 Design Rationale

The following design decisions were made with specific rationales:

Decision 1: Use Object-Oriented Design (OOD) Methodology

Rationale: OOD allows modeling of real-world entities (locations, routes, users) as objects. This improves modularity, reusability, and ease of maintenance. Each object encapsulates both data and behavior.

Decision 2: Adopt Waterfall Process Model

Rationale: The Waterfall model was selected because system requirements are clearly defined from the beginning. Each phase is completed sequentially with proper documentation. This is suitable for a university-level web-based project with stable requirements.

Decision 3: Layered Client-Server Architecture

Rationale: This architecture provides clear separation between presentation, business logic, and data. It allows independent development, easy replacement of frontend, scalability, and security (database never directly exposed to clients).

Alternative Architectures Considered and Rejected:

1. Monolithic single-tier – Poor separation of concerns, difficult to maintain
2. Microservices – Overly complex for project scope
3. Desktop application – Does not meet cross-platform access requirement

Decision 4: PostgreSQL with PostGIS for Database

Rationale: PostGIS is the industry standard for spatial data in open-source applications. It provides efficient spatial indexing for fast route calculations, built-in geospatial functions (ST_Distance, ST_Length), and seamless Django integration.

Decision 5: Dijkstra's Algorithm for Route Calculation

Rationale: Dijkstra's algorithm is well-suited for finding the shortest path in a weighted graph with positive edge weights (distances). It guarantees the optimal solution and performs efficiently on a campus-scale graph (typically under 1000 edges)

3.4 Data Description

The information domain of the Campus Navigation System is transformed into structured database entities using an object-oriented approach. Spatial and non-spatial data (locations, paths, users) are stored in PostgreSQL with PostGIS support. Data is processed through backend services for route calculations and searches.

Data Flow Overview:

User Input → Application Layer → Database Query → PostGIS Processing → Result Set → JSON Serialization → HTTP Response → User Display

Primary Data Entities and Storage:

1. Campus Map (base layer) – Stored on map tile server as raster tiles. Access pattern: Low (cached).
2. POI Locations – Stored in PostgreSQL/PostGIS as Point geometry. Access pattern: High (search/display).
3. Path Network – Stored in PostgreSQL/PostGIS as LineString geometry. Access pattern: Very High (routing).
4. Route Calculations – Stored temporarily in memory as GeoJSON. Not persisted.
5. Admin Credentials – Stored in PostgreSQL as hashed strings. Access pattern: Low (login only).

3.5 Data Dictionary

Alphabetical listing of system entities, their attributes, and methods:

Admin Entity

1. *Attributes:* admin_id (Integer, primary key), username (String, unique), password_hash (String), last_login (DateTime)
2. *Methods:* login() – authenticates admin user; managePOI(action, data) – add/update/delete POI; managePath(action, data) – add/update/delete path

Path Entity

1. *Attributes:* path_id (Integer, primary key), start_node_id (Integer, foreign key), end_node_id (Integer, foreign key), distance (Float, positive), geometry (LineString, PostGIS spatial)
2. *Methods:* calculateDistance() – computes length from geometry; getPathGeometry() – returns GeoJSON representation

POI Entity

1. *Attributes*: poi_id (Integer, primary key), name (String, 100 chars), type (String, 50 chars), latitude (Float, -90 to 90), longitude (Float, -180 to 180)
2. *Methods*: addPOI(name, type, lat, lon) – creates new POI; updatePOI(poi_id, data) – updates existing POI; deletePOI(poi_id) – removes POI

Node Entity

1. *Attributes*: node_id (Integer, primary key), latitude (Float), longitude (Float), is_intersection (Boolean)

Route Entity (Temporary)

1. *Attributes*: route_id (Integer, temporary), total_distance (Float, meters), estimated_time (Float, minutes), geometry (LineString)
2. *Methods*: computeShortestRoute(start, end) – core routing algorithm using Dijkstra

User/Session Entity

1. *Attributes*: user_id (String, session identifier), session_id (String), last_activity (DateTime)
2. *Methods*: searchLocation(keyword) – finds POI by text search; viewMap() – renders initial map; requestRoute(start, end) – triggers route calculation

3.6 Component Design

This section describes the behavior of each system component using procedural description language (PDL) and pseudo code.

Admin.login(username, password)

text

PROCEDURE Admin.login(username, password)

BEGIN

INPUT: username, password

OUTPUT: access_granted (boolean), session_token (string or null)

FETCH admin_record FROM database WHERE username = INPUT_username

IF admin_record EXISTS THEN

IF password MATCHES admin_record.password_hash THEN

session_token = GENERATE_SESSION_TOKEN()

```

        UPDATE last_login = CURRENT_TIMESTAMP
        RETURN (access_granted = TRUE, session_token = session_token)
    ELSE
        RETURN (access_granted = FALSE, session_token = NULL)
    END IF
ELSE
    RETURN (access_granted = FALSE, session_token = NULL)
END IF
END

```

Admin.managePOI(poi_id, action, data)

text

```

PROCEDURE Admin.managePOI(poi_id, action, data)
BEGIN
    INPUT: poi_id, action ('add', 'update', 'delete'), data (name, type, lat, lon)
    OUTPUT: success (boolean), message (string)

    IF action = 'add' THEN
        INSERT INTO map_poi (name, type, latitude, longitude)
        VALUES (data.name, data.type, data.lat, data.lon)
        RETURN (success = TRUE, message = "POI added successfully")

    ELSE IF action = 'update' THEN
        UPDATE map_poi SET name = data.name, type = data.type,
            latitude = data.lat, longitude = data.lon
        WHERE poi_id = INPUT_poi_id
        RETURN (success = TRUE, message = "POI updated successfully")

    ELSE IF action = 'delete' THEN
        DELETE FROM map_poi WHERE poi_id = INPUT_poi_id
        RETURN (success = TRUE, message = "POI deleted successfully")

    ELSE
        RETURN (success = FALSE, message = "Invalid action")
    END IF
END

```

END

User.searchLocation(keyword)

text

PROCEDURE User.searchLocation(keyword)

BEGIN

INPUT: keyword (string)

OUTPUT: matching_locations (array of POI objects)

IF LENGTH(keyword) < 2 THEN

RETURN empty_array

END IF

query_result = EXECUTE_SQL(

"SELECT poi_id, name, type, latitude, longitude

FROM map_poi

WHERE LOWER(name) LIKE LOWER('%' || \$1 || '%')

ORDER BY

CASE

WHEN LOWER(name) = LOWER(\$1) THEN 1

WHEN LOWER(name) LIKE LOWER(\$1 || '%') THEN 2

ELSE 3

END

LIMIT 10",

[keyword]

)

RETURN query_result

END

User.requestRoute(start_point, end_point)

text

PROCEDURE User.requestRoute(start_point, end_point)


```

BEGIN
  INPUT: start_point {lat, lon}, end_point {lat, lon}
  OUTPUT: route_data (geometry, distance, time) OR error

  IF start_point IS NULL OR end_point IS NULL THEN
    RETURN error = "Please select both start and destination points"
  END IF

  route_result = CALL Route.computeShortestRoute(start_point, end_point)

  IF route_result.success = FALSE THEN
    RETURN error = route_result.message
  END IF

  RETURN route_data = {
    geometry: route_result.geometry,
    distance: route_result.total_distance,
    estimated_time: route_result.estimated_time
  }
END

```

Route.computeShortestRoute(start_point, end_point)

text

```

PROCEDURE Route.computeShortestRoute(start_point, end_point)
BEGIN
  INPUT: start_point (lat, lon), end_point (lat, lon)
  OUTPUT: route_geometry (GeoJSON), total_distance (float),
    estimated_time (float), success (boolean)

  // Find nearest nodes to clicked points
  nearest_start = FIND_NEAREST_NODE(start_point)
  nearest_end = FIND_NEAREST_NODE(end_point)

  IF nearest_start IS NULL OR nearest_end IS NULL THEN
    RETURN success = FALSE, message = "No path network near selected points"
  
```

```

END IF

// Build graph from PostGIS
graph = FETCH path_graph FROM PostGIS

// Initialize Dijkstra
distances[all nodes] = INFINITY
distances[nearest_start] = 0
previous[all nodes] = NULL
priority_queue = [(0, nearest_start)]

// Main Dijkstra loop
WHILE priority_queue IS NOT EMPTY
    current = POP smallest distance
    IF current = nearest_end THEN BREAK

    FOR EACH neighbor OF current
        new_distance = distances[current] + edge_weight(current, neighbor)
        IF new_distance < distances[neighbor] THEN
            distances[neighbor] = new_distance
            previous[neighbor] = current
            PUSH (new_distance, neighbor) INTO priority_queue
        END IF
    END FOR
END WHILE

IF distances[nearest_end] = INFINITY THEN
    RETURN success = FALSE, message = "No walking route exists"
END IF

// Reconstruct path
path_nodes = []
current = nearest_end
WHILE current IS NOT NULL
    path_nodes.PREPEND(current)
    current = previous[current]
END WHILE

```

```

// Get geometry for path
route_geometry = GET_PATH_GEOMETRY(path_nodes)
total_distance = distances[nearest_end]
estimated_time = (total_distance / 1.4) / 60 // meters to minutes

RETURN success = TRUE, geometry = route_geometry,
        total_distance = ROUND(total_distance, 1),
        estimated_time = ROUND(estimated_time, 1)
END

```

Path.calculateDistance()

```

text
PROCEDURE Path.calculateDistance(geometry)
BEGIN
    INPUT: geometry (PostGIS LineString)
    OUTPUT: distance_in_meters (float)

    distance_in_meters = EXECUTE_SPATIAL_SQL(
        "SELECT ST_Length($1::geography) as length", [geometry]
    )

    RETURN distance_in_meters
END

```

Chapter 4

Implementation and Testing

4.1 Code Modules

The system is implemented using the following code modules organized by Django app structure:

Main Application (navigation/):

1. settings.py – Django configuration (150 lines)
2. urls.py – Main URL routing (50 lines)
3. wsgi.py – Production entry point (30 lines)

Campus App (campus/):

1. models.py – Database models for POI, Path, Node, Admin (200 lines)
2. views_route.py – Route calculation API (150 lines)
3. views_search.py – Search API (80 lines)
4. views_admin.py – Admin CRUD operations (180 lines)
5. views_auth.py – Login/logout handling (100 lines)
6. serializers.py – JSON serialization (120 lines)
7. urls.py – App-specific routes (40 lines)
8. admin.py – Django admin interface (60 lines)
9. route_algorithm.py – Dijkstra implementation (150 lines)

Frontend Static Files (static/campus/):

1. js/app.js – Main application controller (100 lines)
2. js/map.js – Leaflet map initialization (120 lines)
3. js/route.js – Route calculation and display (150 lines)
4. js/search.js – Search and auto-complete (130 lines)
5. js/admin.js – Admin panel functionality (200 lines)
6. css/styles.css – Custom styling (150 lines)
7. css/responsive.css – Mobile responsive design (80 lines)

Templates (templates/campus/):

1. map.html – Main user interface
2. admin_login.html – Admin login page
3. admin_dashboard.html – Admin data management

4.2 Database Connectivity

The system uses PostgreSQL with PostGIS extension as the primary database. Connectivity is established through Django's ORM.

Connection Settings (settings.py):

1. ENGINE: django.db.backends.postgresql
2. NAME: campus_navigation_db
3. USER: campus_user
4. HOST: localhost (or remote server)
5. PORT: 5432

Connection Process:

1. Django loads database configuration from settings.py at startup
2. Connection pool is established (managed by Django)
3. For standard queries: Django ORM automatically converts Python objects to SQL
4. For spatial queries: Raw SQL with PostGIS functions is used
5. Connection is automatically closed after request completion

PostGIS Initialization (Migration):

sql

```
CREATE EXTENSION IF NOT EXISTS postgis;
```

```
CREATE EXTENSION IF NOT EXISTS postgis_topology;
```

```
CREATE INDEX idx_path_geometry ON map_path USING GIST (geometry);
```

Example Spatial Query:

python

```
def find_nearest_node(lat, lon):
```

```
    cursor.execute("""
```

```
        SELECT node_id, latitude, longitude,
```

```
               ST_Distance(
```

```
               ST_SetSRID(ST_MakePoint(%s, %s), 4326)::geography,
```

```

        ST_SetSRID(ST_MakePoint(longitude, latitude), 4326)::geography

    ) as distance

FROM map_node ORDER BY distance LIMIT 1

"", [lon, lat])

return cursor.fetchone()

```

4.3 Overview of User Interface

User Perspective (Patron)

When a user accesses the Campus Navigation System, the campus map is displayed immediately. Users can zoom using +/- buttons or mouse wheel, and pan by dragging.

To find a location: Users type a building name in the search bar. Auto-complete suggestions appear as they type. Selecting a suggestion highlights that building on the map.

To get directions: Users click on the map to select a starting point (green pin appears), then click again for destination (red pin appears). Clicking "Get Directions" calculates and displays the shortest walking route in blue, along with distance (meters) and estimated time (minutes).

Error handling: The system displays helpful messages for invalid inputs, no route found, or search with no results.

Admin Perspective

Admins navigate to the admin login page (/admin/login/) and enter credentials. After successful login, the admin dashboard shows:

1. POI Management: Table of all POIs with Edit and Delete buttons. "Add New POI" button opens a form for name, type, latitude, longitude.
2. Path Management: Table of paths with edit/delete capabilities.

After each action (add/edit/delete), a success or error message is displayed. The table updates immediately.

4.4 Screen Images

Display screenshots showing the interface from the user's perspective. These can be hand drawn or you can use an automated drawing tool. Just make them as accurate as possible.

(Graph paper works well.)

4.5 Screen Objects and Actions

Object	User Action	System Feedback / Response
Map	Click	Clicked coordinates par pin marker appear hota hai.
Map	Drag	Map smoothly pan (move) hota hai.
Zoom Buttons (+/-)	Click	Map ka scale change hota hai.
Search Bar	Type	500ms ke andar auto-complete suggestions dikhte hain.
Search Suggestion	Select	Map POI par pan hota hai aur marker dikhta hai.

Get Directions Button	Click	Blue route line display hoti hai ya error message aata hai.
Route Info Panel	Auto-update	Distance (m) aur Time (min) ka real-time data dikhta hai.
Admin Login Button	Click	User ko <code>/admin/login/</code> page par redirect kiya jata hai.
Username/Password	Type	Input fields mein characters appear hote hain.
Login Submit	Click	Valid: Dashboard open hota hai; Invalid: Error dikhta hai.
Add POI Button	Click	Naya POI add karne ke liye blank form khulta hai.
Edit POI Button	Click	Form pehle se maujood POI data ke saath khulta hai.

Delete POI Button	Click	Permanent deletion ke liye confirmation dialog aata hai.
Submit Button (Form)	Click	Data save hota hai aur success message display hota hai.
Cancel Button	Click	Form band ho jata hai aur koi changes save nahi hote.
Success Alert	Auto-disappear	Green background aur checkmark icon dikhta hai.
Error Alert	Auto-disappear	Red background aur warning icon dikhta hai.

4.6 Test Cases

The system will be tested using the following key test cases:

Functional Tests:

1. Map loads correctly within 5 seconds
2. Search returns correct POI results with auto-complete within 500ms
3. Route calculation completes within 3 seconds for 95% of requests
4. Route displays correctly with distance and time
5. Same start/destination shows error message
6. Invalid points show appropriate error

Admin Tests:

1. Valid admin login grants access; invalid login shows error
2. Add, edit, and delete POI operations succeed with confirmation
3. Unauthorized access to admin panel redirects to login

Security Tests:

1. SQL injection attempts are blocked
2. XSS attempts are prevented
3. Admin passwords are stored as hashes (not plaintext)
4. Admin session expires after 30 minutes of inactivity

Performance Tests:

1. Map loads within 5 seconds
2. Search auto-complete within 500 milliseconds
3. Route calculation within 3 seconds
4. System handles 25 concurrent route requests

Compatibility Tests:

1. System works on Chrome, Firefox, Edge, and Safari
2. Responsive layout works on desktop, tablet, and mobile devices